Building a RAG application from scratch using LangChain & Pinecone

Loading the environment variables

In [1]:

```
import getpass
import os

def _set_env(var: str):
    if not os.environ.get(var):
        os.environ[var] = getpass.getpass(f"{var}: ")

_set_env("OPENAI_API_KEY")
```

```
OPENAI_API_KEY: ········
```

In [2]:

```
_set_env("PINECONE_API_KEY")
```

```
PINECONE_API_KEY: ········
```

Setting up the model

In [5]:

```
!pip install langchain_openai

from langchain_openai.chat_models import ChatOpenAI

# Retrieve the API key directly from the environment variables when creating the model in
stance
model = ChatOpenAI(openai_api_key=os.getenv("OPENAI_API_KEY"), model="gpt-3.5-turbo")
```

```
Requirement already satisfied: langchain_openai in c:\users\sairam\anaconda3\lib\site-pac
kages (0.1.17)
Requirement already satisfied: langchain-core<0.3.0,>=0.2.20 in c:\users\sairam\anaconda3
\lib\site-packages (from langchain_openai) (0.2.22)
Requirement already satisfied: openai<2.0.0,>=1.32.0 in c:\users\sairam\anaconda3\lib\sit
e-packages (from langchain_openai) (1.37.0)
Requirement already satisfied: tiktoken<1,>=0.7 in c:\users\sairam\anaconda3\lib\site-pac
kages (from langchain_openai) (0.7.0)
Requirement already satisfied: PyYAML>=5.3 in c:\users\sairam\anaconda3\lib\site-packages
(from langchain-core<0.3.0,>=0.2.20->langchain_openai) (6.0)
Requirement already satisfied: jsonpatch<2.0,>=1.33 in c:\users\sairam\anaconda3\lib\site
-packages (from langchain-core<0.3.0,>=0.2.20->langchain_openai) (1.33)
Requirement already satisfied: langsmith<0.2.0,>=0.1.75 in c:\users\sairam\anaconda3\lib\
site-packages (from langchain-core<0.3.0,>=0.2.20->langchain_openai) (0.1.93)
Requirement already satisfied: packaging<25,>=23.2 in c:\users\sairam\anaconda3\lib\site-
packages (from langchain-core<0.3.0,>=0.2.20->langchain_openai) (23.2)
Requirement already satisfied: pydantic<3,>=1 in c:\users\sairam\anaconda3\lib\site-packa
ges (from langchain-core<0.3.0,>=0.2.20->langchain_openai) (1.10.8)
Requirement already satisfied: tenacity!=8.4.0,<9.0.0,>=8.1.0 in c:\users\sairam\anaconda
3\lib\site-packages (from langchain-core<0.3.0,>=0.2.20->langchain_openai) (8.2.2)
Requirement already satisfied: anyio<5,>=3.5.0 in c:\users\sairam\anaconda3\lib\site-pack
ages (from openai<2.0.0,>=1.32.0->langchain_openai) (3.5.0)
Requirement already satisfied: distro<2,>=1.7.0 in c:\users\sairam\anaconda3\lib\site-pac
kages (from openai<2.0.0,>=1.32.0->langchain_openai) (1.9.0)
Requirement already satisfied: httpx<1,>=0.23.0 in c:\users\sairam\anaconda3\lib\site-pac
kages (from openai<2.0.0,>=1.32.0->langchain_openai) (0.27.0)
Requirement already satisfied: sniffio in c:\users\sairam\anaconda3\lib\site-packages (fr
om openai<2.0.0,>=1.32.0->langchain_openai) (1.2.0)
```

```
Requirement already satisfied: tqdm>4 in c:\users\sairam\anaconda3\lib\site-packages (fro
m openai<2.0.0,>=1.32.0->langchain_openai) (4.65.0)
Requirement already satisfied: typing-extensions<5,>=4.7 in c:\users\sairam\anaconda3\lib
\site-packages (from openai<2.0.0,>=1.32.0->langchain_openai) (4.12.2)
Requirement already satisfied: regex>=2022.1.18 in c:\users\sairam\anaconda3\lib\site-pac
kages (from tiktoken<1,>=0.7->langchain_openai) (2022.7.9)
Requirement already satisfied: requests>=2.26.0 in c:\users\sairam\anaconda3\lib\site-pac
kages (from tiktoken<1,>=0.7->langchain_openai) (2.31.0)
Requirement already satisfied: idna>=2.8 in c:\users\sairam\anaconda3\lib\site-packages (
from anyio<5,>=3.5.0->openai<2.0.0,>=1.32.0->langchain_openai) (3.4)
Requirement already satisfied: certifi in c:\users\sairam\anaconda3\lib\site-packages (fr
om httpx<1,>=0.23.0->openai<2.0.0,>=1.32.0->langchain_openai) (2023.7.22)
Requirement already satisfied: httpcore==1.* in c:\users\sairam\anaconda3\lib\site-packag
es (from httpx<1,>=0.23.0->openai<2.0.0,>=1.32.0->langchain_openai) (1.0.4)
Requirement already satisfied: h11<0.15,>=0.13 in c:\users\sairam\anaconda3\lib\site-pack
ages (from httpcore==1.*->httpx<1,>=0.23.0->openai<2.0.0,>=1.32.0->langchain_openai) (0.1
4.0)
Requirement already satisfied: jsonpointer>=1.9 in c:\users\sairam\anaconda3\lib\site-pac
kages (from jsonpatch<2.0,>=1.33->langchain-core<0.3.0,>=0.2.20->langchain_openai) (2.1)
Requirement already satisfied: orjson<4.0.0,>=3.9.14 in c:\users\sairam\anaconda3\lib\sit
e-packages (from langsmith<0.2.0,>=0.1.75->langchain-core<0.3.0,>=0.2.20->langchain_opena
i) (3.9.15)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\sairam\anaconda3\lib\
site-packages (from requests>=2.26.0->tiktoken<1,>=0.7->langchain_openai) (2.0.4)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\sairam\anaconda3\lib\site-p
ackages (from requests>=2.26.0->tiktoken<1,>=0.7->langchain_openai) (1.26.16)
Requirement already satisfied: colorama in c:\users\sairam\anaconda3\lib\site-packages (f
rom tqdm>4->openai<2.0.0,>=1.32.0->langchain_openai) (0.4.6)
```

In [6]:

```
model
```

Out[6]:

```
ChatOpenAI(client=<openai.resources.chat.completions.Completions object at 0x000002688AA4
CE90>, async_client=<openai.resources.chat.completions.AsyncCompletions object at 0x00000
2688AA54C10>, openai_api_key=SecretStr('**********'), openai_proxy='')
```

Test the model by asking a simple question

In [7]:

```
model.invoke("Name 3 yellow spring flowers.")
```

Out[7]:

```
AIMessage(content='1. Daffodil\n2. Forsythia\n3. Tulip', response_metadata={'token_usage'
: {'completion_tokens': 17, 'prompt_tokens': 14, 'total_tokens': 31}, 'model_name': 'gpt-
3.5-turbo-0125', 'system_fingerprint': None, 'finish_reason': 'stop', 'logprobs': None},
id='run-da7f5e56-088c-439d-b8a0-274f059a29d6-0', usage_metadata={'input_tokens': 14, 'out
put_tokens': 17, 'total_tokens': 31})
```

Extract this answer by chaining the model with an output parser. So we use a simple StrOutputParser to extract the answer as a string.

In [8]:

```
from langchain_core.output_parsers import StrOutputParser

parser = StrOutputParser()

chain = model | parser
chain.invoke("Name 3 yellow spring flowers.")
```

Out[8]:

```
'1. Daffodil\n2. Forsythia\n3. Tulip'
```

Introducing prompt templates

We want to provide the model with some context and the question. Prompt templates are a simple way to define and reuse prompts.

In [9]:

```
from langchain.prompts import ChatPromptTemplate

template = """
Answer the question based on the context below. If you can't
answer the question, reply "I don't know".

Context: {context}

Question: {question}
"""

prompt = ChatPromptTemplate.from_template(template)
prompt.format(context="Varsha lives in Bangalore", question="Where does Varsha live?")
```

Out[9]:

```
'Human: \nAnswer the question based on the context below. If you can\'t\nanswer the quest
ion, reply "I don\'t know".\n\nContext: Varsha lives in Bangalore\n\nQuestion: Where does
Varsha live?\n'
```

chain the prompt with the model and the output parser

In [10]:

```
chain = prompt | model | parser
chain.invoke({
    "context": "Varsha lives in Bangalore",
    "question": "Where does Varsha live?"
})
```

Out[10]:

```
'Varsha lives in Bangalore.'
```

Combining chains

We can combine different chains to create more complex workflows. creating a new prompt template for the translation chain

In [11]:

```
translation_prompt = ChatPromptTemplate.from_template(
    "Translate {answer} to {language}"
)
```

creating a new translation chain that combines the result from the first chain with the translation prompt.

In [12]:

```
from operator import itemgetter

translation_chain = (
    {"answer": chain, "language": itemgetter("language")} | translation_prompt | model |
parser
)

translation_chain.invoke(
    {
        "context": "Varsha likes Ice-cream. Varsha's brother name is Badal.",
        "question": "How many siblings does Varsha have?",
        "language": "Spanish",
```

```
    }
)
```

Out[12]:

```
'Varsha tiene un hermano, Badal.'
```

Loading the transcription file

Read the transcription and display the first few characters to ensure everything works as expected

In [13]:

```
with open("transcription.txt") as file:
    transcription = file.read()

transcription[:102]
```

Out[13]:

```
"- I think it's possible that physics has exploits and we should be trying to find them.
Arranging some"
```

Using the entire transcription as context

If we try to invoke the chain using the transcription as context, the model will return an error because the context is too long. Large Language Models support limitted context sizes. so we need to find a different solution (Splitting).

In [14]:

```
try:
    chain.invoke({
        "context": transcription,
        "question": "Is reading papers a good idea?"
    })
except Exception as e:
    print(e)
```

```
Error code: 400 - {'error': {'message': "This model's maximum context length is 16385 tok
ens. However, your messages resulted in 49091 tokens. Please reduce the length of the mes
sages.", 'type': 'invalid_request_error', 'param': 'messages', 'code': 'context_length_ex
ceeded'}}
```

Splitting the transcription

Since we can't use the entire transcription as the context for the model, a potential solution is to split the transcription into smaller chunks. We can then invoke the model using only the relevant chunks to answer a particular question. Let's start by loading the transcription in memory

In [15]:

```
from langchain_community.document_loaders import TextLoader

loader = TextLoader("transcription.txt")
text_documents = loader.load()
```

In [16]:

```
type(text_documents)
```

Out[16]:

```
list
```

There are many different ways to split a document. For this example, we'll use a simple splitter that splits the

There are many different ways to split a document. For this example, we'll use a simple splitter that splits the document into chunks of a fixed size. split the transcription into chunks of 1000 characters with an overlap of 20 characters.

In [17]:

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=20)
documents = text_splitter.split_documents(text_documents)
```

In [18]:

```
type(documents[0])
```

Out[18]:

```
langchain_core.documents.base.Document
```

In [19]:

```
documents[15]
```

Out[19]:

```
Document(metadata={'source': 'transcription.txt'}, page_content='But that seems crazy \'c
ause how many single-cell organisms are there?\nAnd how much time you have surely it\'s n
ot that difficult. And a billion years is not even that long\nof a time really, just all
these bacteria under constrained resources battling it out.\nI\'m sure they can invent mo
re complex. I don\'t understand. It\'s like how to move from a "Hello, World!" program\nt
o invent a function or something like that. I don\'t- - Yeah.\n- Yeah, so I\'m with you.
I just feel like I don\'t see any, if the origin of life, that would be my intuition, tha
t\'s the hardest thing.\nBut if that\'s not the hardest thing \'cause it happened so quic
kly, then it\'s gotta be everywhere and yeah, maybe we\'re just too dumb to see it.\n- We
ll, it\'s just we don\'t have really good mechanisms for seeing this life.\nSo I\'m not a
n expert just to preface this but just from what- - On aliens? I wanna meet an expert on
alien intelligence')
```

In [20]:

```
len(documents)
```

Out[20]:

```
230
```

Constructing the model with openai embeddings

we need to find the relevant chunks from the transcription to send to the model. Here is where the idea of embeddings comes into play. To provide with the most relevant chunks, we can use the embeddings of the question and the chunks of the transcription to compute the similarity between them. We can then select the chunks with the highest similarity to the question and use them as the context for the model

Generate Embeddings

In [21]:

```
from langchain_openai.embeddings import OpenAIEmbeddings
model_name = "text-embedding-3-small"
embeddings = OpenAIEmbeddings(
    model = model_name,
    openai_api_key = os.environ.get("OPENAI_API_KEY")
)
```

Setting up a Vector Store

Constructing the vector store with pinecone

We need an efficient way to store document chunks, their embeddings, and perform similarity searches at scale. To do this, we'll use a vector store. A vector store is a database of embeddings that specializes in fast similarity searches.

Connecting the vector store to the chain We can use the vector store to find the most relevant chunks from the transcription to send to the model. Here is how we can connect the vector store to the chain

Setting up Pinecone

The first step is to create a Pinecone account, set up an index, get an API key, and set it as an environment variable PINECONE_API_KEY.

In [22]:

```
!pip install langchain_pinecone --user
```

```
Requirement already satisfied: langchain_pinecone in c:\users\sairam\appdata\roaming\pyth
on\python311\site-packages (0.1.3)
Requirement already satisfied: aiohttp<4.0.0,>=3.9.5 in c:\users\sairam\anaconda3\lib\sit
e-packages (from langchain_pinecone) (3.9.5)
Requirement already satisfied: langchain-core<0.3,>=0.1.52 in c:\users\sairam\anaconda3\l
ib\site-packages (from langchain_pinecone) (0.2.22)
Requirement already satisfied: numpy<2,>=1 in c:\users\sairam\anaconda3\lib\site-packages
(from langchain_pinecone) (1.24.3)
Requirement already satisfied: pinecone-client<6.0.0,>=5.0.0 in c:\users\sairam\appdata\r
oaming\python\python311\site-packages (from langchain_pinecone) (5.0.0)
Requirement already satisfied: aiosignal>=1.1.2 in c:\users\sairam\anaconda3\lib\site-pac
kages (from aiohttp<4.0.0,>=3.9.5->langchain_pinecone) (1.2.0)
Requirement already satisfied: attrs>=17.3.0 in c:\users\sairam\anaconda3\lib\site-packag
es (from aiohttp<4.0.0,>=3.9.5->langchain_pinecone) (22.1.0)
Requirement already satisfied: frozenlist>=1.1.1 in c:\users\sairam\anaconda3\lib\site-pa
ckages (from aiohttp<4.0.0,>=3.9.5->langchain_pinecone) (1.3.3)
Requirement already satisfied: multidict<7.0,>=4.5 in c:\users\sairam\anaconda3\lib\site-
packages (from aiohttp<4.0.0,>=3.9.5->langchain_pinecone) (6.0.2)
Requirement already satisfied: yarl<2.0,>=1.0 in c:\users\sairam\anaconda3\lib\site-packa
ges (from aiohttp<4.0.0,>=3.9.5->langchain_pinecone) (1.8.1)
Requirement already satisfied: PyYAML>=5.3 in c:\users\sairam\anaconda3\lib\site-packages
(from langchain-core<0.3,>=0.1.52->langchain_pinecone) (6.0)
Requirement already satisfied: jsonpatch<2.0,>=1.33 in c:\users\sairam\anaconda3\lib\site
-packages (from langchain-core<0.3,>=0.1.52->langchain_pinecone) (1.33)
Requirement already satisfied: langsmith<0.2.0,>=0.1.75 in c:\users\sairam\anaconda3\lib\
site-packages (from langchain-core<0.3,>=0.1.52->langchain_pinecone) (0.1.93)
Requirement already satisfied: packaging<25,>=23.2 in c:\users\sairam\anaconda3\lib\site-
packages (from langchain-core<0.3,>=0.1.52->langchain_pinecone) (23.2)
Requirement already satisfied: pydantic<3,>=1 in c:\users\sairam\anaconda3\lib\site-packa
ges (from langchain-core<0.3,>=0.1.52->langchain_pinecone) (1.10.8)
Requirement already satisfied: tenacity!=8.4.0,<9.0.0,>=8.1.0 in c:\users\sairam\anaconda
3\lib\site-packages (from langchain-core<0.3,>=0.1.52->langchain_pinecone) (8.2.2)
Requirement already satisfied: certifi>=2019.11.17 in c:\users\sairam\anaconda3\lib\site-
packages (from pinecone-client<6.0.0,>=5.0.0->langchain_pinecone) (2023.7.22)
Requirement already satisfied: pinecone-plugin-inference==1.0.2 in c:\users\sairam\appdat
a\roaming\python\python311\site-packages (from pinecone-client<6.0.0,>=5.0.0->langchain_p
inecone) (1.0.2)
Requirement already satisfied: pinecone-plugin-interface<0.0.8,>=0.0.7 in c:\users\sairam
\anaconda3\lib\site-packages (from pinecone-client<6.0.0,>=5.0.0->langchain_pinecone) (0.
0.7)
Requirement already satisfied: tqdm>=4.64.1 in c:\users\sairam\anaconda3\lib\site-package
s (from pinecone-client<6.0.0,>=5.0.0->langchain_pinecone) (4.65.0)
Requirement already satisfied: typing-extensions>=3.7.4 in c:\users\sairam\anaconda3\lib\
site-packages (from pinecone-client<6.0.0,>=5.0.0->langchain_pinecone) (4.12.2)
Requirement already satisfied: urllib3>=1.26.0 in c:\users\sairam\anaconda3\lib\site-pack
ages (from pinecone-client<6.0.0,>=5.0.0->langchain_pinecone) (1.26.16)
Requirement already satisfied: jsonpointer>=1.9 in c:\users\sairam\anaconda3\lib\site-pac
kages (from jsonpatch<2.0,>=1.33->langchain-core<0.3,>=0.1.52->langchain_pinecone) (2.1)
Requirement already satisfied: orjson<4.0.0,>=3.9.14 in c:\users\sairam\anaconda3\lib\sit
e-packages (from langsmith<0.2.0,>=0.1.75->langchain-core<0.3,>=0.1.52->langchain_pinecon
e) (3.9.15)
Requirement already satisfied: requests<3,>=2 in c:\users\sairam\anaconda3\lib\site-packa
```

```
Requirement already satisfied: requests<3,>=2 in c:\users\sairam\anaconda3\lib\site-packa
ges (from langsmith<0.2.0,>=0.1.75->langchain-core<0.3,>=0.1.52->langchain_pinecone) (2.3
1.0)
Requirement already satisfied: colorama in c:\users\sairam\anaconda3\lib\site-packages (f
rom tqdm>=4.64.1->pinecone-client<6.0.0,>=5.0.0->langchain_pinecone) (0.4.6)
Requirement already satisfied: idna>=2.0 in c:\users\sairam\anaconda3\lib\site-packages (
from yarl<2.0,>=1.0->aiohttp<4.0.0,>=3.9.5->langchain_pinecone) (3.4)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\sairam\anaconda3\lib\
site-packages (from requests<3,>=2->langsmith<0.2.0,>=0.1.75->langchain-core<0.3,>=0.1.52
->langchain_pinecone) (2.0.4)
```

Load the transcription documents into Pinecone by constructing the index and adding name to it.

In [24]:

```
from pinecone import Pinecone, ServerlessSpec
pc = Pinecone(api_key=os.environ.get("PINECONE_API_KEY"))
# Creates an index using the API key stored in the client 'pc'.
pc.create_index(
    name= "ram-rag-index",
    dimension=1536,
    metric="cosine",
    spec=ServerlessSpec(
        cloud= 'aws',
        region= 'us-east-1'
    )
)
```

connecting to index

In [25]:

```
# connect to index
index = pc.Index("ram-rag-index")
# view index stats
index.describe_index_stats()
```

Out[25]:

```
{'dimension': 1536,
 'index_fullness': 0.0,
 'namespaces': {},
 'total_vector_count': 0}
```

In [26]:

```
from langchain_pinecone import PineconeVectorStore
pinecone =PineconeVectorStore.from_documents(
    documents, embeddings, index_name= 'ram-rag-index'
)
```

In [27]:

```
pinecone
```

Out[27]:

```
<langchain_pinecone.vectorstores.PineconeVectorStore at 0x2688ab589d0>
```

Convert the vector store into a retriever

Run a similarity search on pinecone to make sure everything works

In [28]:

```
pinecone.similarity_search("what is tesla's data engine?")[:3]
```

Out[28]:

```
[Document(metadata={'source': 'transcription.txt'}, page_content="that the team is doing
```

to make sure it all fits and utilizes the engine. So I think it's extremely good engineering\nand then there's all kinds of little insights peppered in on how to do it properly.
- Let's actually zoom out\nTesla's Data Engine\n'cause I don't think we talked about the data engine, the entirety of the layouts of this idea\nthat I think is just beautiful with humans in the loop. Can you describe the data engine?\n- Yeah, the data engine is what I call the almost biological feeling process\nby which you perfect the training sets for these neural networks.\nSo because most of the programming now is in the level of these data sets and make sure they're large, diverse and clean, basically you have a data set\nthat you think is good, you train your neural net, you deploy it, and then you observe how well it's performing\nand you're trying to always increase the quality of your data set. So you're trying to catch scenarios,"),
Document(metadata={'source': 'transcription.txt'}, page_content="Leaving Tesla\nand engineering all of it from the data engine to the human side, all of it.\nCan you speak to why you chose to leave Tesla? - Basically, as I described, I think over time\nduring those five years I've kind of gotten myself into a little bit of a managerial position.\nMost of my days were meetings, and growing the organization, and making decisions about,\nhigh-level strategic decisions about the team and what it should be working on, and so on. And it's kind of like a corporate executive role.\nAnd I can do it. I think I'm okay at it. But it's not like fundamentally what I enjoy. And so I think when I joined,\nthere was no computer vision team because Tesla was just going from the transition of using MobilEye, a third-party vendor, for all of its computer vision\nto having to build its computer vision system. So when I showed up, there were two people training deep neural networks.\nAnd they were training them at a computer at their legs, like down, there was a work."),
Document(metadata={'source': 'transcription.txt'}, page_content="and playing with the basic intuitions. It's like Einstein, Fineman, were all really good at this kind of stuff.\nLike small example of a thing, to play with it, to try to understand it. So that, and obviously now with Tesla,\nyou helped build a team of machine learning\nengineers and a system that actually accomplishes something in the real-world. So given all that, what was the soul searching like?\n- Well, it was hard because obviously, I love the company a lot, and I love Elon, I love Tesla,\nso it was hard to leave. I love the team, basically. But yeah, I think I actually,\nI really potentially interested in revisiting it, maybe coming back at some point, working in Optimus,\nworking in AGI at Tesla. I think Tesla's going to do incredible things. It's basically a massive large-scale robotics\nkind of company with a ton of in-house talent for doing real incredible things. And I think human robots are going to be amazing.")]

setup the new chain using Pinecone as the vector store

In [29]:

```
from langchain_core.runnables import RunnablePassthrough

chain = (
    {"context": pinecone.as_retriever(), "question": RunnablePassthrough()}
    | prompt
    | model
    | parser
)
chain.invoke("What is Tesla's Data Engine?")
```

Out[29]:

'The Tesla Data Engine is described as a process by which training sets for neural networks are perfected, with a focus on ensuring the data sets are large, diverse, and clean. The process involves training neural networks with the data sets, deploying them, observing their performance, and constantly improving the quality of the data sets.'

In [ ]:

```
```